

# FORCme Standard Operating Procedure (SOP)

Maxwell Brown, Institute for Rock Magnetism, University of Minnesota

April 17, 2026

## 1 Purpose

This SOP describes how to run the current FORCme notebook workflow for a single FORC file, a stacked set of repeated FORC files, and a batch of independent FORC files.

## 2 Prerequisites and installation guidance

Before running the notebook workflow, the user should ensure that Python and the required scientific packages are installed, including `numpy`, `matplotlib`, `scipy`, and `numba`. `Numba` is strongly recommended because it compiles the most computationally expensive LOESS routines to machine code at runtime and substantially improves processing speed for dense FORC grids and larger datasets.

It is recommended that `PmagPy` is installed and that `forc_funcs.py` is placed in the `PmagPy/pmagpy` directory, with that directory available on the Python path. This arrangement allows the user to save and run the notebook from any location, rather than needing to keep a copy of the module in every working directory. In practice, this means a separate notebook can be saved for each specimen, stack, or study workflow while preserving the exact processing parameters and analysis choices used for that run.

This approach is also helpful from a FAIR and reproducibility perspective. A user can retain one notebook per analysis, archive those notebooks alongside the study, and, if desired, upload the full notebook set as part of the digital record for a project so that each processing decision is transparent and recoverable.

## 3 Directory and file preparation

- **Individual mode** (`mode="i"`): provide a single FORC text file.
- **Stack mode** (`mode="s"`): place repeated FORC files for one specimen in a single directory.
- **Batch mode** (`mode="b"`): place all files to be processed independently in a single directory.
- Use `file_type`, for example `"*.txt"`, to control which files are matched.

## 4 Notebook cell 1: run the main pipeline

```
import numpy as np
import importlib
```

```
import forc_funcs as ff
importlib.reload(ff)

ff.set_plot_style("Arial")
```

## 4.1 Individual mode

```
out = ff.run_forc_pipeline(
    mode="i",
    path="myfile.txt",
    file_type="*.txt",
    do_regrid=True,
    B_step=0.001,
    regrid_method="linear",
    regrid_extrapolate=False,
    smooth_strength=1.0,
    min_pts_strength=1.0,
    color_scale_version=2,
    show_contours=True,
    plot_hyst=True,
    plot_fraction=0.1,
    plot_hbha=True,
    Hu_min=None,
    Hu_max=None,
    Hc_min=None,
    Hc_max=None,
    replace_first=True,
    replace_last=True,
    do_lower_branch_subtract=True,
    plot_lower_subtracted_hyst=False,
    normalize_to_unit=True,
    pct=100,
    display_upsample_factor=0,
)
```

## 4.2 Stack mode

```
out = ff.run_forc_pipeline(
    mode="s",
    path="my_stack_directory",
    file_type="*.txt",
    stack_method="median",
    do_regrid=True,
    B_step=0.001,
    regrid_method="linear",
    regrid_extrapolate=False,
    smooth_strength=1.0,
    min_pts_strength=1.0,
    color_scale_version=2,
    show_contours=True,
    plot_hyst=True,
    plot_fraction=0.1,
    plot_hbha=True,
    Hu_min=None,
```

```

Hu_max=None,
Hc_min=None,
Hc_max=None,
replace_first=True,
replace_last=True,
do_lower_branch_subtract=True,
plot_lower_subtracted_hyst=False,
normalize_to_unit=True,
pct=100,
display_upsample_factor=0,
)

```

### 4.3 Batch mode

```

out = ff.run_forc_pipeline(
    mode="b",
    path="my_batch_directory",
    file_type="*.txt",
    do_regrid=True,
    B_step=0.001,
    regrid_method="linear",
    regrid_extrapolate=False,
    smooth_strength=1.0,
    min_pts_strength=1.0,
    color_scale_version=2,
    show_contours=True,
    plot_hyst=True,
    plot_fraction=0.1,
    plot_hbha=True,
    Hu_min=None,
    Hu_max=None,
    Hc_min=None,
    Hc_max=None,
    replace_first=True,
    replace_last=True,
    do_lower_branch_subtract=True,
    plot_lower_subtracted_hyst=False,
    normalize_to_unit=True,
    pct=100,
    display_upsample_factor=0,
)

```

## 5 Notebook cells 2 and 3 - profile calculation

After the main FORC distribution has been calculated, the notebook can be used to generate one-dimensional profiles through the  $(B_u, B_c)$  representation of  $\rho$ . These profiles are intended to help describe the position, amplitude, and width of FORC features in a more quantitative way than can be read directly from the color plot alone.

An additional one-dimensional smoothing option, controlled through `smooth.sigma_bins`, may be applied to the extracted profiles. This affects only the appearance and stability of the profile curves themselves; it does not recalculate the underlying two-dimensional FORC distribution. The

profile workflow therefore separates the calculation of  $\rho$  from the later task of documenting and visualizing specific cuts through that distribution.

## 5.1 Automated profiles

In the **automatic** profile mode, the code first identifies the bounded peak  $\rho$  value within the displayed ( $B_u, B_c$ ) window. It then constructs two orthogonal profiles through that point:

- a **horizontal profile**, showing  $\rho$  as a function of  $B_c$  at the selected  $B_u$ , and
- a **vertical profile**, showing  $\rho$  as a function of  $B_u$  at the selected  $B_c$ .

These default profiles provide a consistent way to document the main FORC feature in a specimen, including the position of the peak and the general spread of the signal along the coercivity and interaction axes.

## 5.2 Custom profiles

In many cases, however, the feature of interest is *not* located exactly at the maximum  $\rho$  value. For example, a user may wish to examine a shoulder, a secondary ridge, a low-coercivity contribution, or another part of the distribution that is scientifically important but not dominant in amplitude. For this reason, the code also provides a **custom** profile mode. In this mode, the user specifies the target  $B_u$  and  $B_c$  values directly, and the program extracts the corresponding horizontal and vertical cuts through that chosen location. This allows the user to interrogate a specific part of the FORC diagram rather than being restricted to the global peak.

## 5.3 Export

For documentation and export, the profile routines write out both the numerical profile data and the associated profile figures. In this way, the user retains a record of exactly which cuts were taken, whether they were selected automatically from the bounded peak or manually through the custom mode, and how those cuts were displayed for later interpretation, publication, or archiving.

# 6 Automatic sample titles

If `sample.title` is not provided:

- `mode="i"` uses the file name without extension.
- `mode="b"` uses each file name without extension.
- `mode="s"` uses the common part of the matched file names.

# 7 Output locations

Outputs are written beneath the input directory:

- `FORC_figures`: main FORC figures.
- `FORC_profiles`: profile text files and profile figures.
- A MagIC export file is made for the raw data when mode "i" or "b" is run. The user is advised to check over this file and adapted for their own lab where necessary.

## 8 Recommended user settings

- Use `stack_method="median"` when repeated files may contain outliers.
- Start with `display_upsample_factor=0`. Increase to 2 or 4 only for smoother plotting if desired.
- Use `do_regrid=True` for consistency across samples or if the source export is not already on a regular field grid.
- Use lower-branch subtraction only when justified by the measurement protocol.

## 9 Troubleshooting

- **No files matched:** check `path` and `file_type`.
- **Too many NaNs in  $\rho$ :** reduce smoothing requirements or inspect data sparsity and segmentation.
- **Unexpected stack title:** use `sample_title` explicitly if the default common-file-name logic is not ideal.
- **Plots too blocky or too smooth:** adjust `display_upsample_factor`.